

University of Cape Town

Bayesian Computational Methods

Tutorial 1

Prior Elicitation for Bayesian Generalised Linear Models

Author:
Isabella Lethbridge

Student Number:
LTHISA001

August 23, 2026

Introduction

We are fitting the following Bayesian logistic regression model to the `SAheart` dataset from the `ElemStatLearn` R package:

$$log(\frac{p_i}{1-p_i}) = \beta_0 + \beta_1 \text{age} + \beta_2 \text{famhist} + \beta_3 \text{tobacco},$$

where $p_i = \Pr(chd_i = 1 \mid \boldsymbol{x}_i, \boldsymbol{\beta})$ and $\boldsymbol{x}_i = (1, \text{age}_i, \text{famhist}_i, \text{tobacco}_i)$ and `famhist` is coded as a binary predictor where 0 indicates absence and 1 indicates presence.

Question a

Expert 1 provided the median probability that a patient would develop coronary heart disease for M=14 design points. Based on the methods laid out in Choy et al. (2009), we denote the median probabilities as Z_m and model them with the following Beta distribution:

$$\text{Z}_m \sim Beta(\mu_m, \gamma_m) \quad \text{for} \quad m = 1, \ldots, \text{M},$$

where μ_m is the mean and γ_m is the precision parameter. The logit link below relates the expert's elicited median probabilities to the regression coefficients.

$$logit(\mu_m) = \boldsymbol{x}_m^T \beta, \quad \boldsymbol{x}_m^T = (1, \text{age}_m, \text{famhist}_m, \text{tobacco}_m).$$

Under the assumption that the prior distribution of the regression coefficients are multivariate normal random variables:

$$\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2, \beta_3)^T \sim N_4(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0).$$

$\boldsymbol{\mu}_0$ and $\boldsymbol{\Sigma_0}$ can be estimated by fitting a Beta regression model. `betareg`, with a default logit link function was fitted to the 14 elicited median probabilities using covariates `age`, `famhist` and `tobacco`.

Following the Choy et al. (2009) elicitation procedure, we use the following sample estimates to specify the prior distribution parameters:

$$\boldsymbol{\mu}_0 = \hat{\boldsymbol{\beta}} \quad \text{and} \quad \boldsymbol{\Sigma}_0 = \widehat{Var}(\hat{\boldsymbol{\beta}}).$$

Extracting only the first 4 rows and columns from $\widehat{Var}(\hat{\boldsymbol{\beta}})$ isolates the regression parameters from the nuisance parameter, ϕ.

Question b

Expert 2 provided lower and upper quartile values associated with the probability that a patient would develop coronary heart disease, across M=14 design points.

Let $p_m = \Pr(chd_m = 1 \mid \boldsymbol{x}_m, \boldsymbol{\beta})$, then:

$$\Pr(\theta_m \leq q_{0.25,m}) = 0.25, \quad \Pr(\theta_m \leq q_{0.75,m}) = 0.75.$$

Following the methods outlined in Hosack et al. (2017), the 25% and 75% quantiles are transformed to linear predictors via the logit link function:

$$\eta_{0.25,m} = \text{logit}(q_{0.25,m}) \quad \text{and} \quad \eta_{0.75,m} = \text{logit}(q_{0.75,m}).$$

Assuming a normal prior for $\boldsymbol{\beta}$:

$$\eta_m = \boldsymbol{x}_m^T \boldsymbol{\beta} \sim N(m_m, v_m)$$

This gives:

$$\Pr(g(p_m) \leq g(q_{0.25,m})) = \Phi(\eta_m; m_m, v_m) = 0.25.$$

Assuming that $\eta_m = \boldsymbol{x}_m^T \boldsymbol{\beta} \sim N(m_m, v_m)$, the target interval probabilities are $\boldsymbol{l} = (l_1, l_2, l_3)^T = (0.25, 0.50, 0.25)^T$. For each design point m, the parameters (m_m, v_m) are estimated to minimise the Kullback-Leibler (KL) divergence using `nlm()`:

$$\mathrm{KL}(m_m, v_m) = \sum_{j=1}^{3} \log\Big(\frac{l_j}{Z_j(m_m, v_m)}\Big) l_j.$$

where

$$Z_1(m_m, v_m) = \Phi\left(\frac{\eta_{0.25,m} - m_m}{\sqrt{v_m}}\right),$$

$$Z_2(m_m, v_m) = \Phi\left(\frac{\eta_{0.75,m} - m_m}{\sqrt{v_m}}\right) - Z_1(m_m, v_m),$$

$$Z_3(m_m, v_m) = 1 - \Phi\left(\frac{\eta_{0.75,m} - m_m}{\sqrt{v_m}}\right).$$

Let $\boldsymbol{m} = (m_1, \ldots, m_M)^\top$ and $\boldsymbol{V} = \mathrm{diag}(v_1, \ldots, v_M)$. The prior mean vector $\boldsymbol{\mu}_0$ and covariance matrix $\boldsymbol{\Sigma}_0$ for Expert 2 are obtained as follows:

$$\boldsymbol{\mu}_0 = \left(\boldsymbol{X}^\top \boldsymbol{V}^{-1} \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{V}^{-1} \boldsymbol{m} \quad \text{and} \quad \boldsymbol{\Sigma}_0 = \left(\boldsymbol{X}^\top \boldsymbol{V}^{-1} \boldsymbol{X}\right)^{-1}.$$

Question c

Table 1: Elicited prior mean and standard deviations from expert 1 and expert 2.

	Expert 1 mean	Expert 2 mean	Expert 1 sd	Expert 2 sd
Intercept	-3.9041	-3.6984	0.0971	0.4876
age	0.0525	0.0466	0.0028	0.0144
famhist	0.9964	1.1246	0.0462	0.2313
tobacco	0.1114	0.1145	0.0044	0.0298

The two elicitation methods result in similar prior mean vectors, as seen from the β coefficients, in Table 1. The differences in the prior standard deviations indicate the differing levels of uncertainty associated with the two approaches. Expert 1 produces a more concentrated prior compared to Expert 2, as seen in Figure 1. The standard deviations associated with Expert 2's prior are 5 times larger than that of Expert 1's.

Figure 1 illustrates how Expert 1's prior has greater precision. This would result in the prior dominating the regression analysis by pulling the posterior distribution towards

its means irrespective of the actual data. Expert 2's prior has less influence. This would allow for the observed data to influence the posterior distribution.

`betareg` estimates the precision parameter based on overall model fit across the design points. A high model fit can falsely inflate precision, producing an overly concentrated prior. The Kullback-Leibler divergence captures the expert's uncertainty since the variance for each design point is individually optimised to fit the experts elicited quantiles.

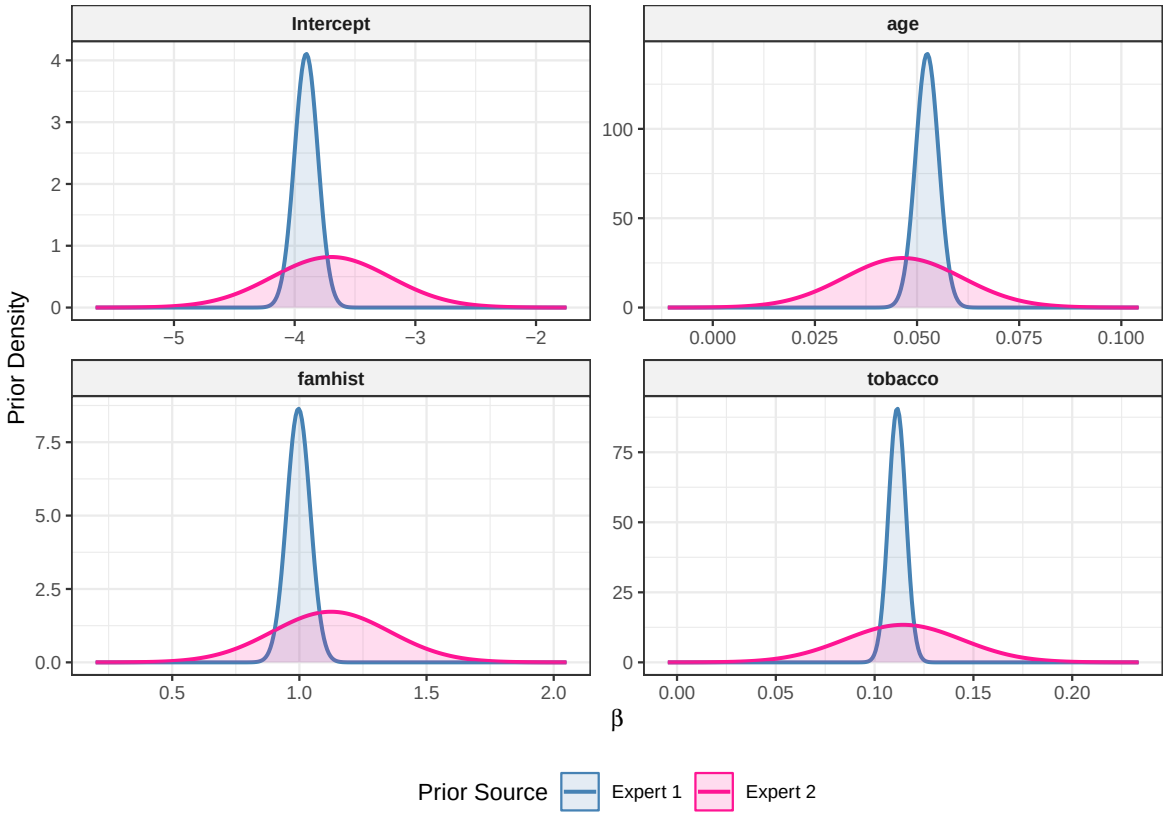


Figure 1: Comparison of marginal prior distributions for Expert 1 and Expert 2.

Appendix

```r
# Set up:
# Load data:
require(ElemStatLearn)
data('SAheart')
# View(SAheart)

# Random training data set:
set.seed(1)
training = SAheart[sample(1:462,size=300),]

# Load required libraries:
library(betareg)
library(ggplot2)
library(dplyr)
library(purrr)

# Question a:
# Table 1: Design points
age     = c(20,29,32,34,40,41,42,43,46,54,55,61,64,64)
famhist = c(0,0,0,0,0,1,0,0,0,1,1,1,1,1)
tobacco = c(0.00,1.50,0.00,0.40,1.62,0.90,2.92,6.75,
            12.00,7.60,4.80,25.01,8.60,12.18)
E1_med  = c(0.05,0.11,0.1,0.12,0.165,0.33,0.2,0.275,
            0.46,0.68,0.64,0.96,0.8,0.86)

# Store data in a dataframe:
E1_df = data.frame(
   Z       = E1_med,
   age     = age,
   famhist = famhist,
   tobacco = tobacco
)

# Fit a beta regression model:
betareg_fit = betareg(Z~age+famhist+tobacco,data=E1_df)

# Obtain mu0:
mu0_a = coef(betareg_fit,model='mean')

# Obtain Sigma0:
S0_a = vcov(betareg_fit)[1:4,1:4]

# Check correlations:
cor_mat_a = cov2cor(S0_a)

# Question b:
# Table 1: Design points
E2_25 = c
   (0.05,0.08,0.08,0.10,0.14,0.32,0.18,0.25,0.39,0.65,0.59,0.93,0.77,0.82)

E2_75 = c
   (0.075,0.13,0.125,0.13,0.18,0.39,0.22,0.31,0.5,0.74,0.68,0.98,0.83,0.9)
```

```r

# No. observations:
n = length(E2_25)

# Define logit function:
logit_fxn = function(x) {
  log(x/(1-x))
}

# Transform fractiles:
p25 = logit_fxn(E2_25)
p75 = logit_fxn(E2_75)

# Kullback-Leibler function:
# This what we wish to optimize
KL_fxn = function(par,p25,p75) {
  # Mean and standard deviation:
  m = par[1]
  v = par[2]
  sd = sqrt(v)

  # Calculate integrals:
  Z1 = pnorm(q=p25,mean=m,sd=sd)
  Z2 = pnorm(q=p75,mean=m,sd=sd)-Z1
  Z3 = 1-pnorm(q=p75,mean=m,sd=sd)

  # Calculate Kullback-Leibler for each 'integral':
  KL1 = sum(log(0.25/Z1)*0.25)
  KL2 = sum(log(0.50/Z2)*0.50)
  KL3 = sum(log(0.25/Z3)*0.25)

  # Return a single Kullback-Leibler value:
  KL = sum(KL1,KL2,KL3)
  return(KL)
}

m_vec = numeric(n)
v_vec = numeric(n)

# For loop to optimize m and v for each observation:
for (i in 1:14) {
  init_m = 0
  init_v = 1

  nlm_fit = nlm(f   =KL_fxn,
                p   =c(init_m,init_v),
                p25 = p25[i],
                p75 = p75[i]
                )

  m_vec[i] = nlm_fit$estimate[1]
  v_vec[i] = nlm_fit$estimate[2]
}
```

```r

# Question c:
# Predictor names:
param_names = c('Intercept','age','famhist','tobacco')

# Prior standard deviations:
sd_a = sqrt(diag(S0_a))
sd_b = sqrt(diag(S0_b))

# Prior comparison:
prior_comparison = data.frame(
  Parameter    = param_names,
  Expert1_Mean = as.numeric(mu0_a),
  Expert2_Mean = as.numeric(mu0_b),
  Expert1_SD   = sd_a,
  Expert2_SD   = sd_b
)

# Display table:
prior_comparison = as.data.frame(round(prior_comparison[, -1],4))
colnames(prior_comparison) = c('Expert 1 mean',
                               'Expert 2 mean',
                               'Expert 1 sd',
                               'Expert 2 sd')
rownames(prior_comparison) = param_names

# knitr::kable(
#   prior_comparison
# )

# Plot prior distributions:
mu_E1 = as.vector(mu0_a)
sd_E1 = sqrt(diag(S0_a))
mu_E2 = as.vector(mu0_b)
sd_E2 = sqrt(diag(S0_b))

# Generate density evaluation data for each parameter:
plot_data = map_dfr(1:4, function(i) {
  # Define bounds:
  x_min  = min(mu_E1[i]-4*sd_E1[i],mu_E2[i]-4*sd_E2[i])
  x_max  = max(mu_E1[i]+4*sd_E1[i],mu_E2[i]+4*sd_E2[i])
  x_grid = seq(x_min,x_max,length.out=300)

  tibble(
    Parameter = factor(param_names[i],
                       levels=param_names),
    x = rep(x_grid, 2),
    Density = c(dnorm(x_grid,mean=mu_E1[i],sd=sd_E1[i]),
                dnorm(x_grid,mean=mu_E2[i],sd=sd_E2[i])),
    Expert = rep(c("Expert 1", "Expert 2"), each = 300)
  )
})

# Plot overlaid density curves:
```

```
# ggplot(plot_data,
#        mapping=aes(x = x,
#                    y = Density,
#                    color = Expert,
#                    fill = Expert)) +
#   geom_line(linewidth = 0.8) +
#   geom_area(alpha = 0.15,
#             position = "identity") +
#   facet_wrap(~ Parameter,
#              scales = "free",
#              ncol = 2) +
#   scale_color_manual(values = c("Expert 1" = "steelblue",
#                                 "Expert 2" = "deeppink")) +
#   scale_fill_manual(values = c("Expert 1" = "steelblue",
#                                "Expert 2" = "deeppink")) +
#   labs(
#     x = expression(beta),
#     y = "Prior Density",
#     color = "Prior Source",
#     fill = "Prior Source"
#   ) +
#   theme_bw(base_size = 10) +
#   theme(
#     legend.position = "bottom",
#     strip.background = element_rect(fill = "gray95"),
#     strip.text = element_text(face = "bold")
#   )
```